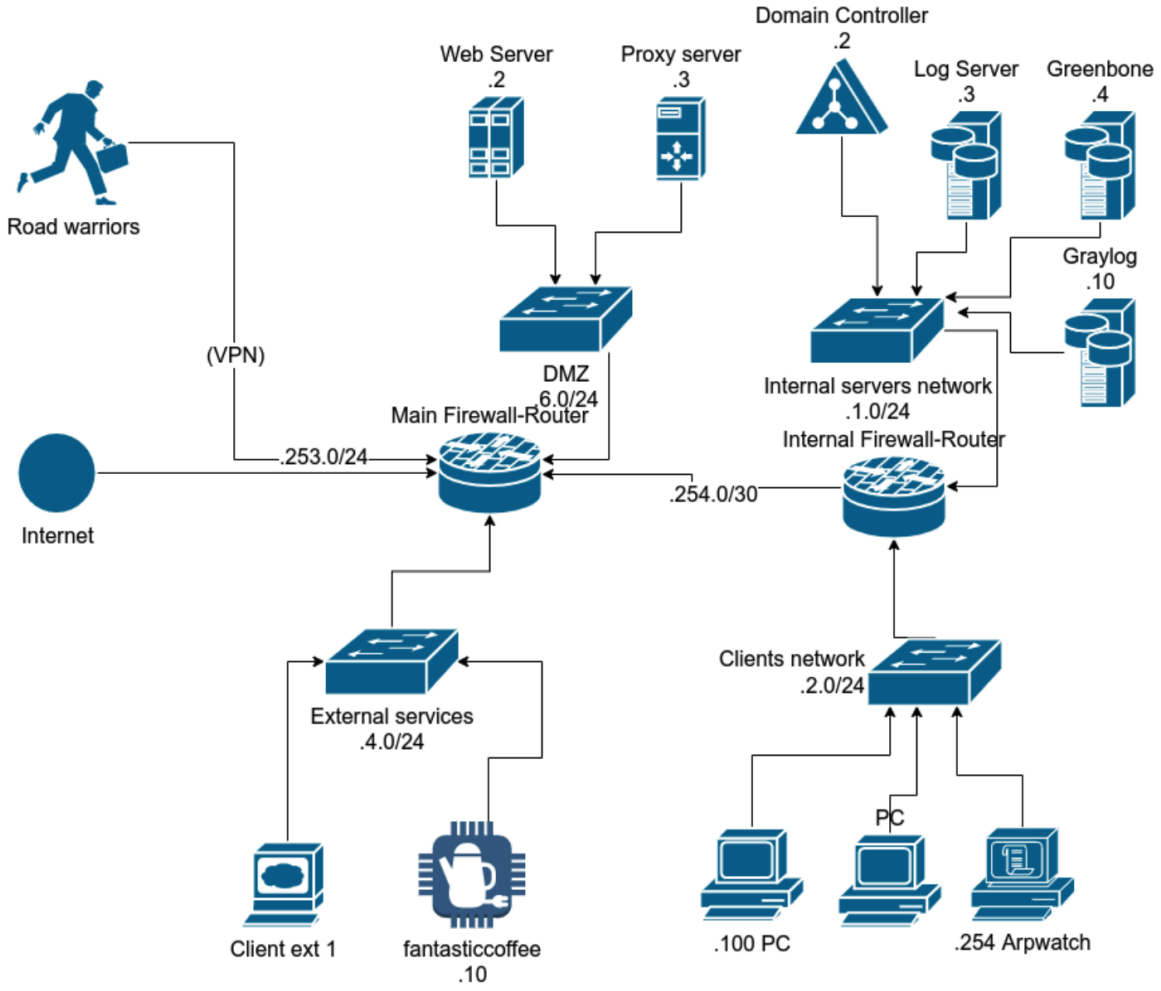
ACME A1 REPORT

Group number: 09

Milillo Vincenzo 2215260
Zucconelli Giorgio 1966894
Altomare Andrea 2199451
Ruocco Salvatore 2222155

Practical Network Defense - Assignment 1

Topology


Domain Controller
.2
Web Server
.2
Proxy server
.3
Log Server
.3
Greenbone
.4
Road warriors
Graylog
.10
(VPN)
DMZ
6.0/24
Internal servers network
.1.0/24
Main Firewall-Router
Internal Firewall-Router
.253.0/24
.254.0/30
Internet
External services
.4.0/24
Clients network
.2.0/24
PC
Client ext 1
fantasticcoffee
.10
.100 PC
.254 Arpwatch

Device Table

Device Name	IP Address	Network
Road Warriors	any	Internet
Main Firewall-Router (289)	100.100.0.1 / 100.100.6.1 / 100.100.4.1 / 100.100.254.1	WAN .0.1 / ROUTER LAN .254.1 / EXTERNAL .4.1 / DMZ .6.1
Web Server (297)	100.100.6.2	DMZ (100.100.6.0/24)
Proxy Server (298)	100.100.6.3	DMZ (100.100.6.0/24)
External Client 1 (299)	100.100.4.100	External Services (100.100.4.0/24)
FantasticCoffee (300)	100.100.4.10	External Services (100.100.4.0/24)
Internal Firewall-Router (290)	100.100.254.2 / 100.100.1.1 / 100.100.2.1	ROUTER LAN .254 / INTERNAL.s.n .1.1 / CLIENT.n .2.1
DNS Server (293)	100.100.1.2	Internal Servers (100.100.1.0/24)
Log Server (294)	100.100.1.3	Internal Servers (100.100.1.0/24)
Greenbone (296)	100.100.1.4	Internal Servers (100.100.1.0/24)
Graylog (301)	100.100.1.10	Internal Servers (100.100.1.0/24)
Client PC 1	100.100.2.x	Client Network (100.100.2.0/24)
Client PC 2	100.100.2.x	Client Network (100.100.2.0/24)
Arpwatch (292)	100.100.2.254 or 250 (?)	Client Network (100.100.2.0/24)

Legenda	Confirmed devices	Device not present in proxmox environment

Network/LAN Overview

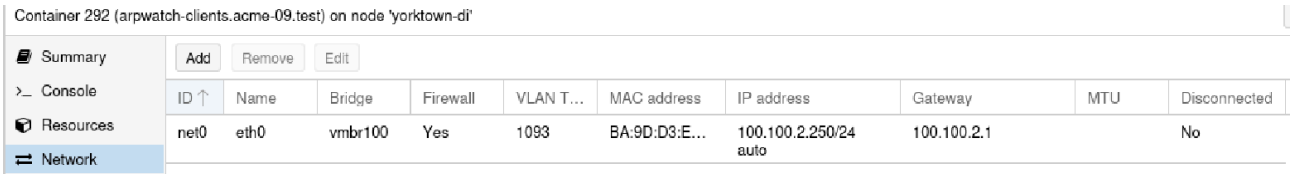
LAN/Network	Subnet	Connected To
Internet	100.64.253.0/24	Main Firewall
DMZ	100.64.6.0/24	Main Firewall
External Services	100.64.4.0/24	Main Firewall
Firewalls LAN	100.64.254.0/30	Main & Internal Firewall
Internal Servers Network	100.64.1.0/24	Internal Firewall
Client Network	100.64.2.0/24	Internal Firewall

Initial brainstorming

We have to set some rules on the two firewalls in the ACME network. We have to do this with OPNsense, a FreeBSD-based firewall and routing software.

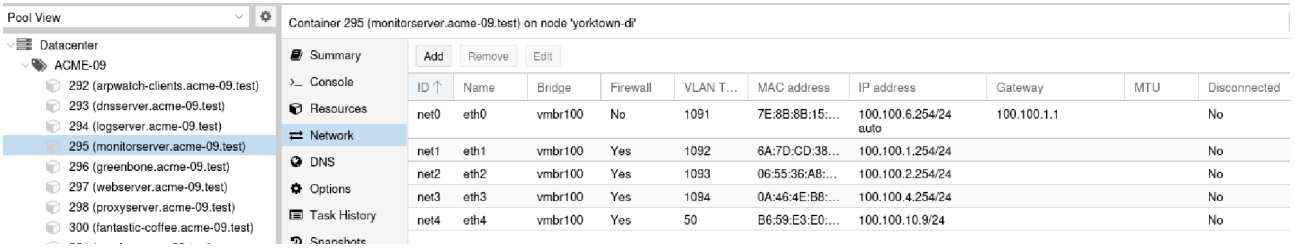
Looking at the host's IP configuration under > Network:

Arpwatch seems to have IP address .250 instead of .254 (as in topology).



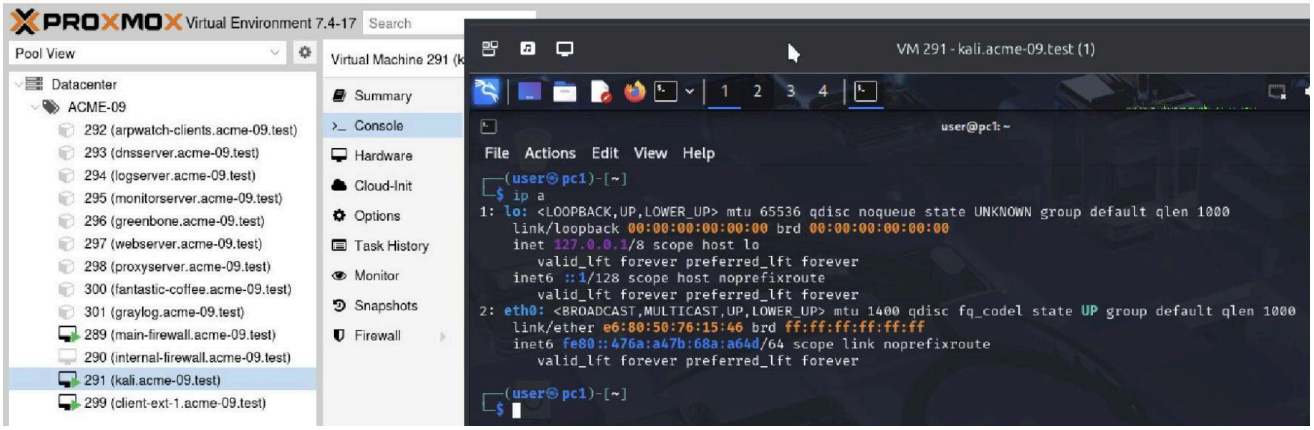
In the ACME environment there is a device (n.295) called monitorserver:

It isn't shown in the ACME topology, it has interfaces on many different LANs



monitorserver is a hidden internal device that connects to multiple LANs across the ACME network. Although it is not represented in the official topology, it plays a crucial role in monitoring network traffic and supervising the activities within the lab environment. It passively observes the network without interacting directly with the other systems configured by students. This device should be left untouched and excluded from any configuration or management activities.

Similarly there is also a device (n.291) called kali.acme that has only an ipv6 address:



kali.acme is a special-purpose machine that has only an IPv6 address and does not appear in the official ACME network topology. Its presence suggests it may be used for future exercises focused on IPv6 protocol testing, penetration testing, or red team simulations. Since it is not part of the current assignment, students are instructed to ignore its presence and avoid making any modifications or connections to this system.

Every other device seems to be regularly present.

As suggested by the professor, to allow our host and the client to interact with the main firewall:

Firewall > Interfaces > WAN > untick Block private/bogon networks > Save > Apply (!!)

OPTIONAL (TODO) - Firewall rules to allow our host to interact with the firewall:

Firewall > Rules > WAN > + (add rule) > (set the rule) > Apply (!!)

- The interface is WAN, the direction is IN, TCP/IP Version is IPv4, protocol TCP, Source Single host or Network in this case 100.101.0.0/24, Destination in this case is the Web Server so we select Single host or Network and insert 100.100.6.2 /32 (single host), the Destination Port is HTTP, Category "From the VPN net" and a Description like "allow HTTP traffic to the WebSERVER".
  (I did not save/apply this rule since it was just for demonstration purposes)
- If we want to create a similar rule but for the firewall we can select, clone and change the Destination to "This Firewall" and set the Destination Port to ANY (this is quite convenient but I am using host 299).

Some students noticed that some files in certain virtualized ACME machines are overwritten after each reboot (like the /etc/resolv.conf file in the dns server host). You can prevent these files from being overwritten by creating a special file that makes proxmox to ignore them: it is sufficient to create a file with prefix .pve-ignore in the same directory, like /etc/.pve-ignore.resolv.conf.

For configuring the firewalls, connect from client-ext-1 at the following addresses:

- main: 100.100.4.1
- internal: 100.100.2.1

Here is a sketch of the steps to perform:

- Request from main a DHCPv6 address + prefix
- Configure the main's interfaces to announce SLAAC using the "Tracking interface" option
- Configure the main's INTERNAL interface to run the DHCPv6 server and delegate a /60 prefix

- Configure the internal interfaces to announce SLAAC using the "Tracking interface" option
- Modify both firewall rules also to allow any IPv6 traffic
- Verify IPv6 connectivity in the whole network
- (optional - TODO) Modify host configurations to generate GUAs with the "stable privacy" technique instead of EUI-64
- Install "dnsmasq" in the dns server host
- Configure dnsmasq(**) to manage the acme-XX.test domain using the /etc/hosts file
- Fill the /etc/hosts file with the addresses of all the hosts in the network
- Verify that everything is working with DNS names

**: to solve the "race condition" between dnsmasq and the SLAAC at boot, you should add the following lines in the /etc/sysctl.conf of dns server:

- net.ipv4.ip_nonlocal_bind = 1
- net.ipv6.ip_nonlocal_bind = 1

In order to configure IPV6 in our environment we have to use DHCP with Prefix Delegation, we have to request /56 address, the idea is that when we use IPV6 networks we usually have a 64 bit long addresses so we are left with 8 bits for subnetting purposes, in order to have /56 we have 3 clusters of 16 bits and the last one divided in two bytes (each of 8 bits). The first 3 clusters starting from the left are provided by the internet service provider or ISP, 8 bits are used to store the number of the ACME in hex (in our case 09 -> 09**|****) while the last 8 are used in fact for subnetting.

How do we assign IP subnets to our environment?

We will split the IP address space between the main and internal networks, since we have a /56 the network will be split in 0900, 0940, 0980, 09B0. The half with 0900, 0940 will be assigned to the main firewall, 0980, 09B0 to the internal firewall. In order for everything to be compatible and related to the IPV4 addresses we will also match the numbers that we have in IPV4, one last thing to recall is that the two spaces will be /60, but the network used in IPV6 are /64.

The only thing that will change between devices belonging to the same network 0900/64 is the last bit that can range between 1 and F like so: 0901-090F. The same is valid for all the networks above.

DMZ - 0906, EXT.CLT - 0904, F.LAN - 090F, INT.S - 0981, INT.CL.N -0982. (0-7 main, 8-F int).

We have to replicate the delegation process of the ISP to the main firewall also into the internal firewall using DHCPv6 and so delegating the /60 that will be 0980 to the internal firewall.

Starting from client-ext-1, we can see that we don't have any IPV6 address but only the link-local:

```
┌──(user㉿pc1)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group default qlen 1000
    link/ether e6:80:50:76:15:46 brd ff:ff:ff:ff:ff:ff
    inet6 fe80::476a:a47b:68a:a64d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

We connect to the main firewall via 100.100.4.1

On the firewall we can see there are only link-local IPV6 addresses so we must set a GUA address for every interface on the different networks, in order to do that we can go to Firewall > Interfaces.

We select WAN and set the IPV6 Configuration Type to DHCPv6 and set the Prefix Delegation Size to /56 then save and apply changes.

Interface	Device	VLAN	Link Type	IPv4	IPv6
WAN (wan)	em0		static	100.100.0.2/24	fd00::7461:8cff:feb4:d112/64 fd00::65bc:70e1:309d:e2c5/128 fe80::7461:8cff:feb4:d112/64

For the DMZ instead we will use a SLAAC, with opnsense we will use the IPV6 Configuration Type track interface so that we can keep the delegation prefix provided by the ISP, then we set the IPV6 Prefix ID to 06 then save and apply changes.

DMZ (lan)	em1		static	100.100.6.1/24	2001:470:b5b8:906:44fd:65ff:fe1b:8895/64 fe80::44fd:65ff:fe1b:8895/64

For the Ext.cl. network interface we do the same as DMZ but with 04 then save and apply changes.

EXTERNAL_CLIENTS (opt1)	em3		static	100.100.4.1/24	2001:470:b5b8:904:d42d:f4ff:fe06:e2d9/64 fe80::d42d:f4ff:fe06:e2d9/64

Finally for the Internal network interface we have to use a different approach, we have to use the Track Interface with DHCP server to delegate the prefix delegation to the internal firewall, then we set the IPV6 Prefix ID to 0F and check the option Allow manual adjustment of DHCPv6 and Router Advertisement, then save and apply changes.

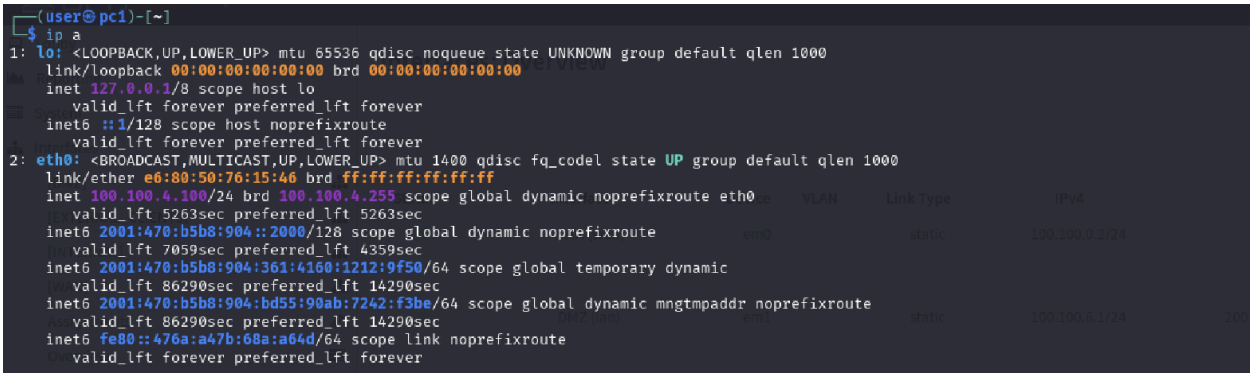
INTERNAL (opt2)	em2		static	100.100.254.1/30	2001:470:b5b8:90f:d0cb:c3ff:fe2b:367e/64 fe80::d0cb:c3ff:fe2b:367e/64

In order to complete the internal network Router Advertisement mechanism we must go under the section called Services > Router Advertisement > Internal and on the Router Advertisement select managed to provide a stateful configuration, we also checked the option Advertise Default Gateway then save and apply changes.

The next step is to configure the DHCPv6 server on Services > ISC DHCPv6 > Internal we can check the Enable option and also set the Range from ::2 to ::2 since we want to provide one single address (to the internal router/firewall), similarly we can set the Prefix Delegation Range (the prefix range assignable from the internal firewall) from ::80 to ::80 since we want to use only that one with /60 size for Prefix delegation (so all the addresses between ::80-::8F), then save and apply changes.

EXTRA CONSIDERATION

If we go on the client-ext-1 terminal and look at the addresses we can see that thanks to Ext.cl. network configuration we received some router advertisements and an IPV6 address was set also for this host automatically.



In detail we received different kinds of addresses:

- The first one is a static IPV6 address.
- The second one is a global temporary dynamic address.
- The third one is the classical UI64 address.
- The fourth one is the link-local one we already had before.
- Now we connect to the internal firewall via 100.100.2.1

First let's have a look at the Interfaces Overview:

Interface	Device	VLAN	Link Type	IPv4	IPv6
EXTERNAL (wan)	em0		static	100.100.254.2/30	fe80::3898:ddff:fe40:9944/64
SERVERS (opt1)	em1		static	100.100.1.1/24	fe80::e4f5:18ff:fe8b:b3e5/64
CLIENTS (lan)	em2		static	100.100.2.1/24	fe80::64a3:2eff:fe83:1e7f/64
Unassigned Interface	enc0				
Loopback (lo0)	lo0		static	127.0.0.1/8	::1/128 fe80::1/64

As we have done for the main firewall we go to Firewall > Interfaces > External and set the IPV6 Configuration Type to DHCPv6 and set the Prefix Delegation Size to /60 then save and apply changes. (The internal will receive the ::2 as set before in the main firewall range for DHCPv6).

EXTERNAL (wan)	em0		static	100.100.254.2/30	2001:470:b5b8:90f::2/128 fe80::3898:ddff:fe40:9944/64

For the Servers network interface we will use the IPV6 Configuration Type track interface so that we can keep the delegation prefix provided by the ISP, then we set the IPV6 Prefix ID to 1 since the internal firewall uses a /60 network only the last byte of the address will change.

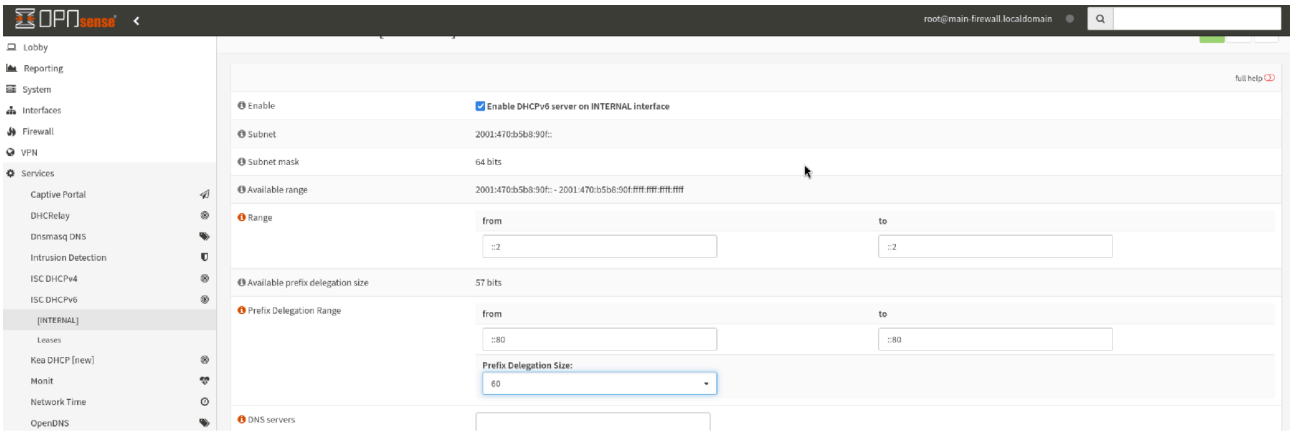
For the Clients network we do exactly the same but with ID 2.

SERVERS (opt1)	em1		static	100.100.1.1/24	2001:470:b5b8:981:e4f5:18ff:fe8b:b3e5/64 fe80::e4f5:18ff:fe8b:b3e5/64
CLIENTS (lan)	em2		static	100.100.2.1/24	2001:470:b5b8:982:64a3:2eff:fe83:1e7f/64 fe80::64a3:2eff:fe83:1e7f/64

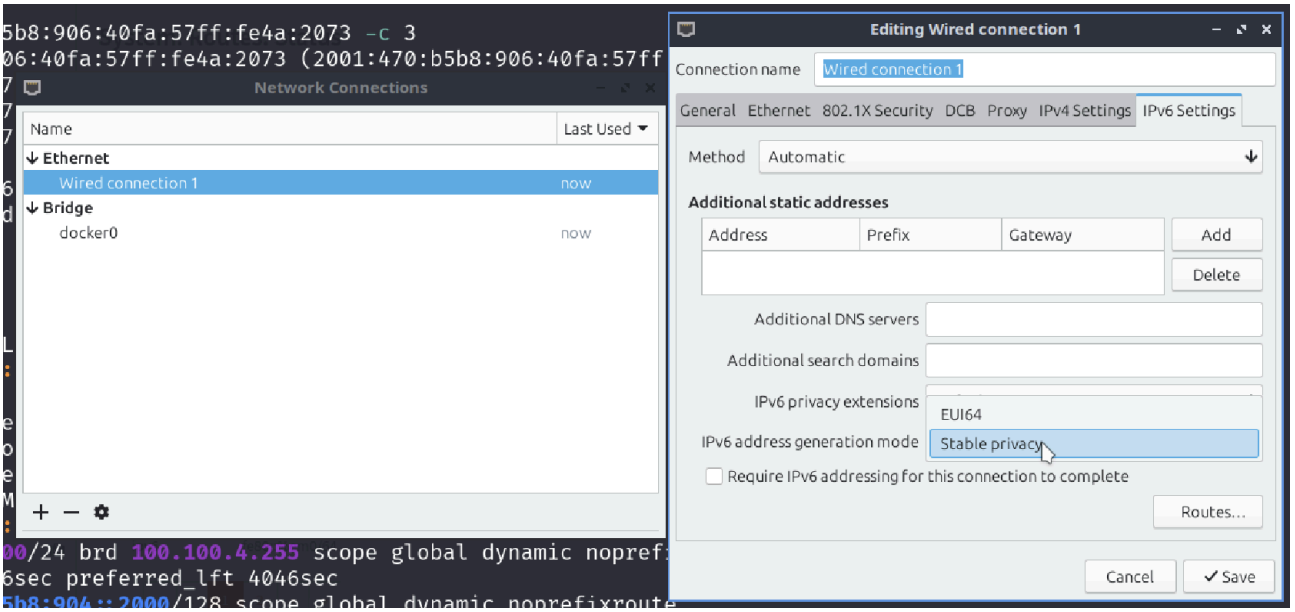
Now we must also allow any IPV6 traffic on our network on both the firewalls.

To do that we must go to Firewall > Rules > Client/External/Servers/Dmz/Internal and change the TCP/IP Version to IPV4+IPV6. (not for WAN).

something went wrong on the internal network → it was the prefix delegation range onPass the main firewall set to /64 instead of /60, fixed it and now pings between internal and external work fine. (also updated data shown before).



One thing we can notice on the external client is that we have two kind of addresses, one is the temporary dynamic address the other one is the EUI64 address, but we can opt for a different kind of connection; we can switch from the UI to the stable privacy that is stable in relation to the prefix we are using but also hides the MAC address, so it's kinda unrelated since it is generated manually with a **pseudo-random function**.




```
inet6 2001:470:b5b8:904:a0f1:f5c0:d993:4921/64 scope global temporary dynamic
   valid_lft 86390sec preferred_lft 14390sec
inet6 2001:470:b5b8:904:bd55:90ab:7242:f3be/64 scope global dynamic mngtmpaddr noprefixroute
   valid_lft 86390sec preferred_lft 14390sec
```

For connecting something on this machine the first address will be used, for receiving packets from the outside we will accept connection on the second one since it is more stable.

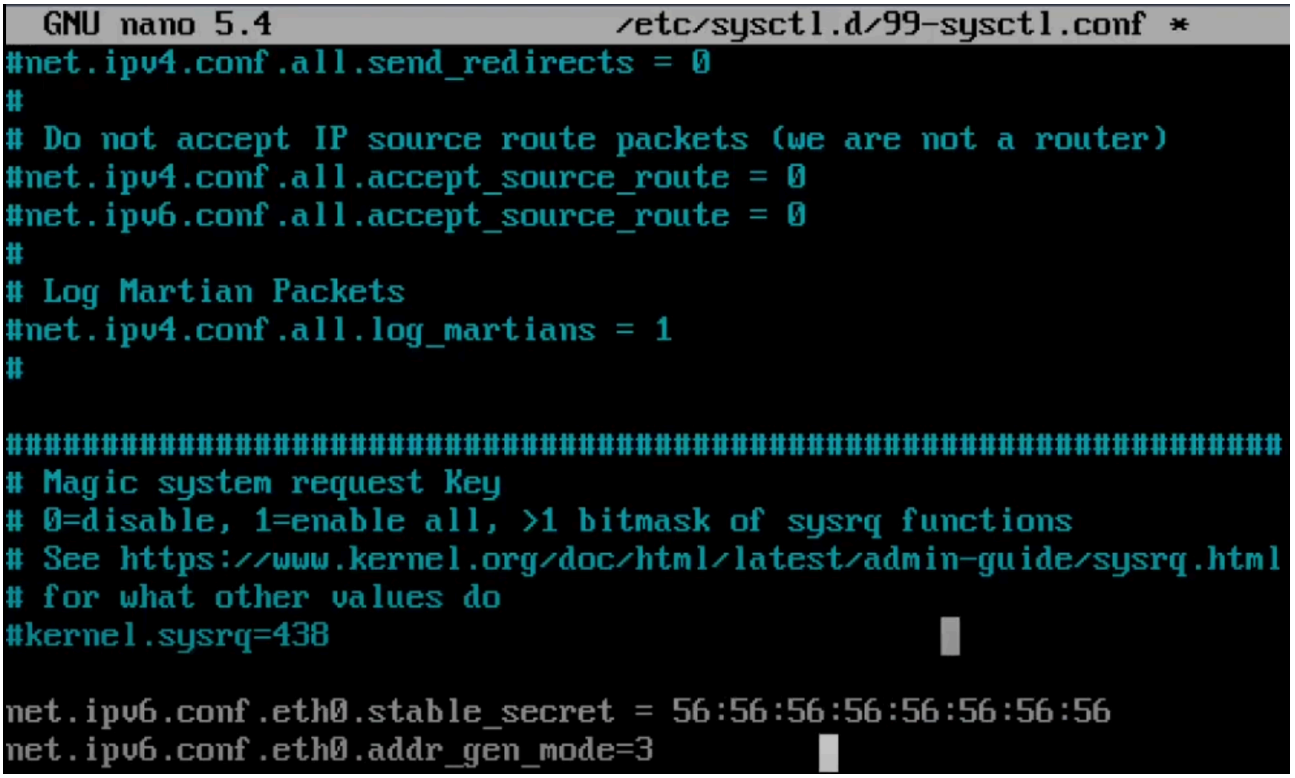
We can do the same on the web server for instance using the sysctl system:

```
root@webserver:~# sysctl -r addr_gen_mode net
net.ipv6.conf.all.addr_gen_mode = 0
net.ipv6.conf.default.addr_gen_mode = 0
net.ipv6.conf.eth0.addr_gen_mode = 0
net.ipv6.conf.lo.addr_gen_mode = 0
root@webserver:~# sysctl -w net.ipv6.conf.eth0.addr_gen_mode=3
net.ipv6.conf.eth0.addr_gen_mode = 3
```

Then we should do as usual ip link set eth0 down and then up.

```
inet6 2001:470:b5b8:906:e782:f997:4839:f7b/64 scope global dynamic mngtmpaddr stable-privacy
   valid_lft 86391sec preferred_lft 14391sec
inet6 fe80::9db0:1ff6:2f36:b3f5/64 scope link stable-privacy
   valid_lft forever preferred_lft forever
```

The address we created is temporary, the address is in fact created using the stable secret, if it is not set it uses a random generated one that changes every reboot, and this is not what we want, so we need to set the address gen mode to 3 but also set the stable secret value, in order to do that we have to edit the file doing nano /etc/sysctl/99-sysctl.conf


GNU nano 5.4 /etc/sysctl.d/99-sysctl.conf *
#net.ipv4.conf.all.send_redirects = 0
#
Do not accept IP source route packets (we are not a router)
#net.ipv4.conf.all.accept_source_route = 0
#net.ipv6.conf.all.accept_source_route = 0
#
Log Martian Packets
#net.ipv4.conf.all.log_martians = 1
#
###
Magic system request Key
0=disable, 1=enable all, >1 bitmask of sysrq functions
See https://www.kernel.org/doc/html/latest/admin-guide/sysrq.html
for what other values do
#kernel.sysrq=438
net.ipv6.conf.eth0.stable_secret = 56:56:56:56:56:56:56:56
net.ipv6.conf.eth0.addr_gen_mode=3

Evaluation of security policy

Before we do anything, we read through all the rules that will need to be set so that we have the big picture of the network. To avoid writing the addresses of each host or sub-network into the firewall rules we have created aliases.

Configuring Services

Web service

Starting from the first point i went on the webserver and created a simple index, then i tested if port 8000 was available:

```
root@webserver:~# echo '<!doctype html><html><body><h1>ACME DMZ Web Service</h1></body
></html>' > ~/index.html
root@webserver:~# cd ~
python3 -m http.server 8000 --bind 0.0.0.0 --directory .
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
^C
Keyboard interrupt received, exiting.
root@webserver:~#
```

and then tested if it was accessible by the client:

```
┌──(root㉿pc1)-[/home/user]
└─# wget http://100.100.6.2:8000

--2025-05-11 17:18:02--  http://100.100.6.2:8000/
Connecting to 100.100.6.2:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 71 [text/html]
Saving to: 'index.html'

index.html                  100%[═══════════════════════════════════════>]      71  --.-KB/s    in 0s

2025-05-11 17:18:02 (9.15 MB/s) - 'index.html' saved [71/71]
```

Then i created a certificate:

```
root@webserver:~# openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout ~/server.pem -out ~/server.pem \
  -subj "/CN=$(hostname)"
Generating a RSA private key
.....................+++++
.....................+++++
writing new private key to '/root/server.pem'
-----
root@webserver:~#
```

DNS

The first step on the dns server is to install the dnsmasq with apt install dnsmasq

The issue we encountered was related to DNS resolution inside the virtual machine dns server. By default, PVE generates the “/etc/resolv.conf” file on each boot with a nameserver that, in our case, points to “151.100.4.132. While this DNS server is technically valid, it was not suitable for resolving common external domains such as “deb.debian.org” or “ftp.debian.org”. As a result, any attempt to install packages using “apt”

or download files with “wget” failed because the system was unable to resolve domain names. To fix this, I leveraged a feature mentioned by the professor: creating a file with the “.pve-ignore.” prefix in the same directory, specifically “/etc/.pve-ignore.resolv.conf”. This file is recognized by Proxmox as a user-defined configuration that should not be overwritten at boot. We created the file with the following contents:

```
root@dnsserver:~# cat /etc/.pve-ignore.resolv.conf
search acme.corp
nameserver 8.8.8.8
nameserver 1.1.1.1
```

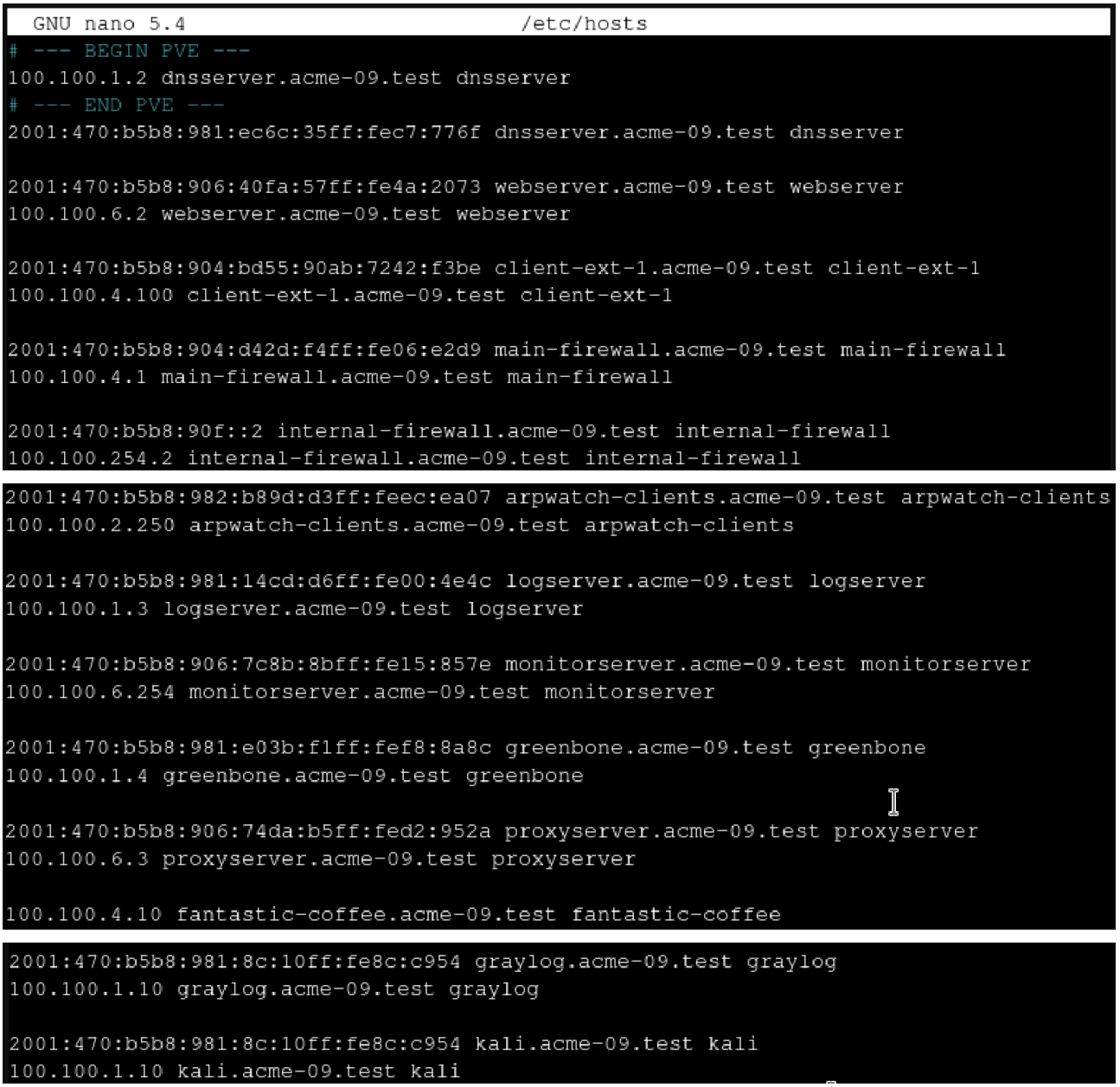
This configuration uses Google's and Cloudflare public DNS servers, which reliably resolve external domain names. Then, we copied this “.pve-ignore.resolv.conf” to “/etc/resolv.conf”, effectively replacing the problematic PVE-generated file.
There is also an option (that i didn’t use) to make it fixed: chattr +i /etc/resolv.conf

After doing this, name resolution started working correctly. We were able to update the package index with “apt update”, download all necessary dependencies, and install “dnsmasq” cleanly.

Then the second step is to copy the initial configuration into a file that we can use later on if we need a backup cp /etc/dnsmasq.conf /etc/dnsmasq.conf.orig, then we modify the file using nano editor:

```
root@dnsserver:~# grep -E "^(#|$)" /etc/dnsmasq.conf -nv
21:bogus-priv
74:local=/acme-09.test/
111:listen-address=100.100.1.2,127.0.0.1,2001:470:b5b8:981:ec6c:35ff:fec7:776f
115:no-dhcp-interface=eth0
124:bind-interfaces
135:expand-hosts
144:domain=acme-09.test
```

Those are the lines that I uncommented and set to make the dnsmasq to work as required.Now we have to collect every IPV4/6 address of the hosts and put them in the file /etc/hosts so that in the future we can use names instead of addresses.



To enable internal DNS resolution across all the machines in the ACME network, I configured each host to use the dns server (100.100.1.2) as its primary name resolver. Since Proxmox tends to overwrite the /etc/resolv.conf file at every reboot, I adopted the suggested workaround by creating a file named /etc/.pve-ignore.resolv.conf containing a custom DNS configuration. Specifically, I included the internal domain search suffix acme-09.test and pointed the nameserver to the internal IP of the dns server. After saving the file, I copied it to /etc/resolv.conf, effectively overriding Proxmox's default behavior. This allowed each machine to consistently resolve names such as webserver.acme-09.test or dnsserver.acme-09.test without relying on external DNS infrastructure.

I applied this procedure to all hosts in the environment, ensuring a uniform configuration across LAN segments. Once done, I verified DNS functionality on each machine by running resolution commands. On most systems, I used tools like host or ping to confirm that names were resolved

correctly to their corresponding IPv4 and IPv6 addresses. This confirmed that dnsmasq on the dns server was functioning correctly and that all clients were pointing to it properly.

However, I encountered a special case with lightweight or container-based systems, such as greenbone. These hosts lacked common tools like host or ping, which made standard DNS testing unavailable. To overcome this, I used the getent hosts command, which queries the system's name service switch configuration and returns results from DNS when properly configured. This method proved effective and reliable for verifying DNS resolution on minimal environments.

Thanks to this consistent and well-planned setup, the entire ACME network is now capable of resolving hostnames internally through a centralized DNS server, with a persistent configuration that survives system reboots.

If after reboot the DNS resolution does not work as expected, the correct behavior can be restored by manually copying the saved configuration using the command: *cp /etc/.pve-ignore.resolv.conf /etc/resolv.conf*

Policy implementation in OPNsense

1. All the ACME hosts must use the internal DNS Server as a DNS resolver.

Done previously in DNS setup.

2. The DNS Server should be able to answer DNS requests coming from the Internet

Dns server is not reachable from the outside. This was because of the main firewall rules for the wan interface.

I created an alias for dns server, web server and proxy server so I can use a single rule in firewalls for both ipv4 and ipv6.

PROBLEM: firewalls were not using the internal dns(100.100.1.2). So aliases did not resolve ip addresses correctly. Changed it in both firewalls settings under system->settings->general.

Created firewall rules to allow udp 53 to dns server and tcp 80, tcp 443 to web server.

On internal_firewall allow requests from local DNS server to 8.8.8.8 and 1.1.1.1

3. The HTTP/HTTPS service provided in the DMZ has to be accessible from the Internet. It can also be accessed using the WAN address of the main firewall.

Webserver http/https have to be accessible using the main firewall wan ip. I added port forwarding (from mainfirewall to the webserver) under firewall->nat->port forward.

Tried to test this but the webserver does not seem to be up listening on port 80/443. Vincenzo created the index.html and was starting it with python3.

Quickly installed apache2 on the webserver so that the web services are available after boot automatically. Also changed /etc/apache2/sites-available/default-ssl.conf to use the keys that Vincenzo generated.

Everything seems to be working correctly.

4. The proxy service provided in the DMZ has to be accessible by the hosts of the ACME network and from the Internet.

Added firewall rule that allows traffic from internet to proxy server. Was not sure which ports a proxy server uses since they can vary so I did not put constraints on ports in the rule leaving option “any”.

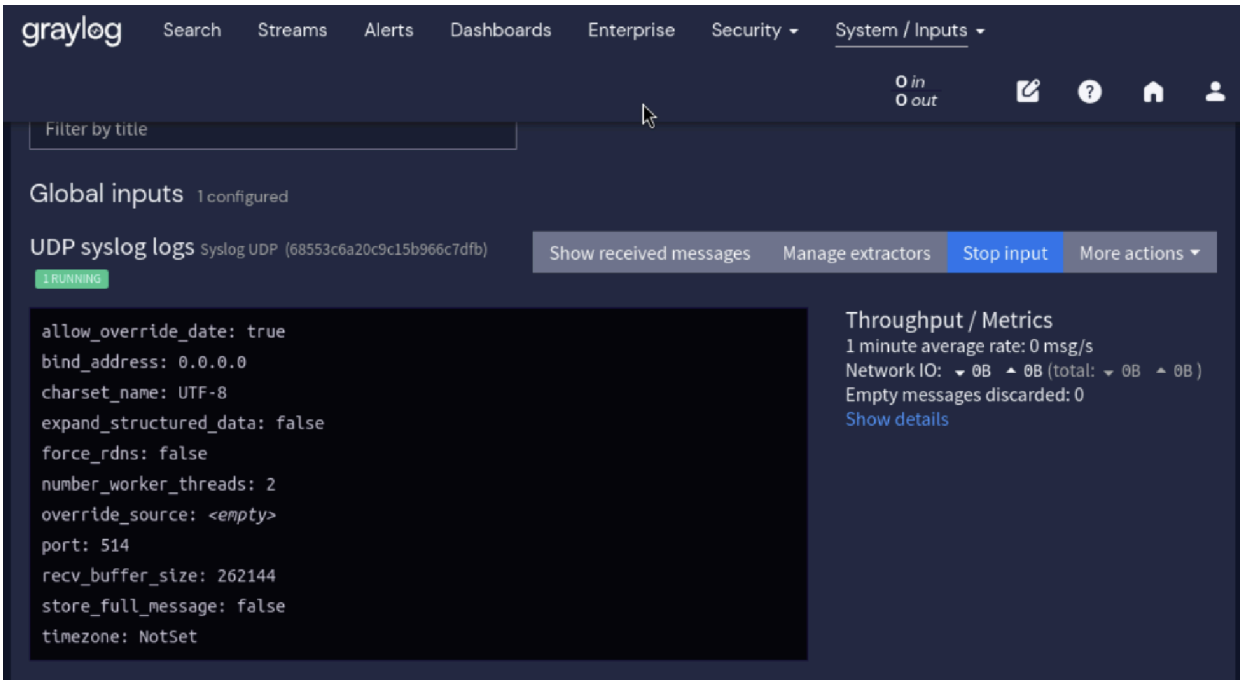
5. Besides the DNS resolver, the other services in the Internal server network must be accessible only to hosts of the Client and DMZ networks

Created an alias in the internal firewall representing the internal servers without the dns server.
Created an alias representing the DMZ.
Created three rules on internal firewall, interface “SERVERS”, direction out:

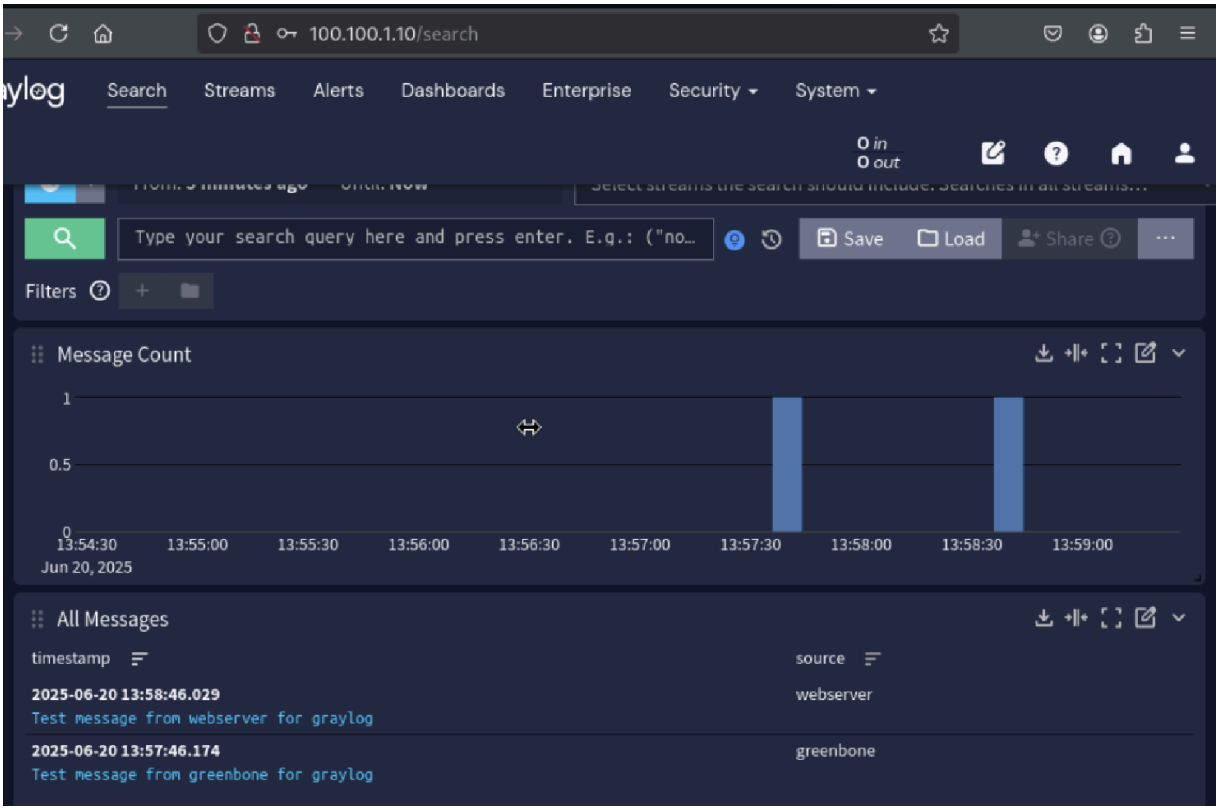
1) allow connection to INT_SERVERS_WITHOUT_DNS from clients network.
2) allow connection to INT_SERVERS_WITHOUT_DNS from DMZ.
3) block every connection to INT_SERVERS_WITHOUT_DNS

6. All the hosts (but the Client network hosts) have to use the syslog and the log collector services on the Log server (syslog) and Graylog server.

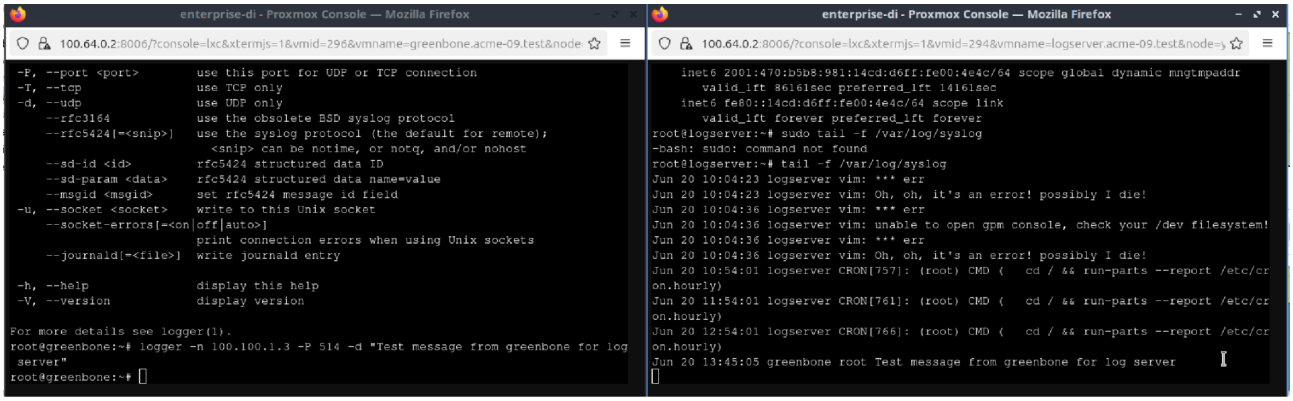
We first configure the Graylog server in order to accept syslog logs using UDP protocol on port 514. We do that by connecting to the GUI through port 80 using the browser. We then create an input going on the tab “System > Inputs”.

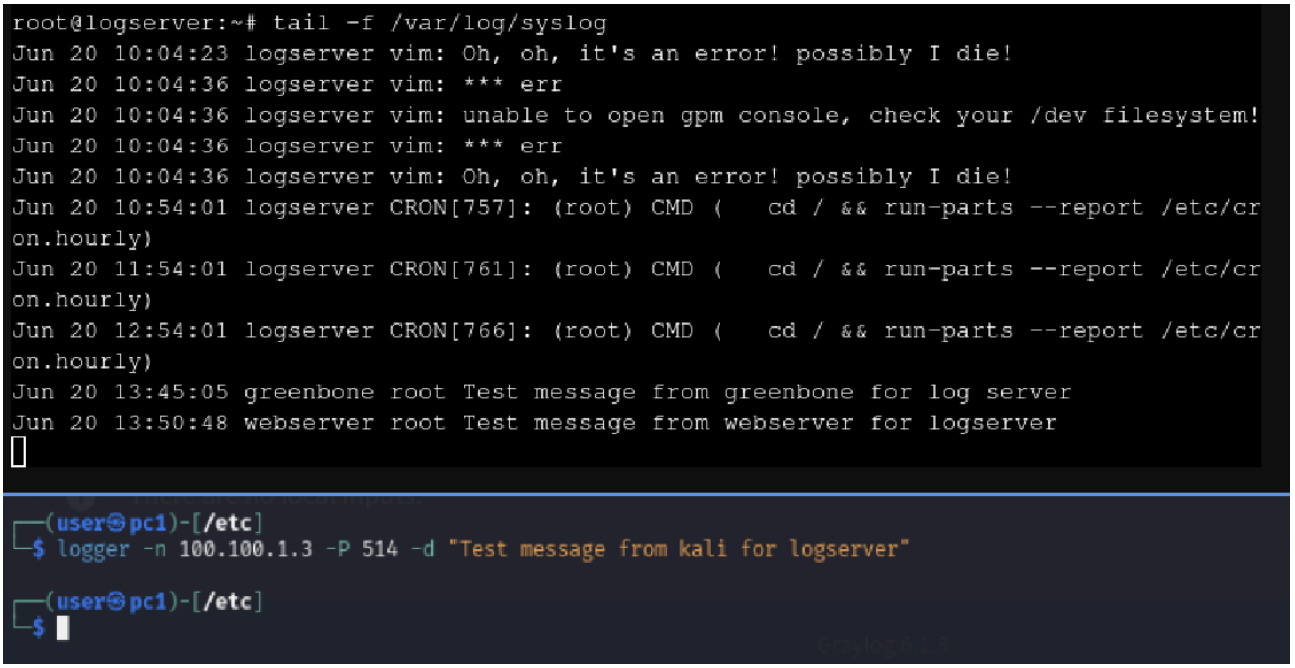


In the internal firewall, we allow for incoming UDP traffic on port 514 to the Graylog server (the Log collector and SIEM). Then, in the main firewall, we allow for UDP traffic on port 514 from the DMZ and the External Services networks to the Graylog server.
We finally test that the logs are correctly sent and received logging in one of the hosts that are allowed to send log to the Graylog server by entering on the terminal:
logger -n 100.100.1.10 -P 514 -d "Test message from green bone for graylog server"



Finally, we allow for UDP traffic on port 514 (that is, the port that syslog uses) for incoming traffic for the Log server (that uses syslog), and we do that for every network except for the Client network. We of course test that everything works properly like above, but sending the logs to the Log server this time:





As we can see from the last screenshot, log messages sent from a host from the Clients network are not allowed to reach the Log server (as for the requirements).

7. The Greenbone server must be able to scan all the network hosts.

In the internal firewall we allow all the traffic coming from Greenbone for any kind of protocol (since vulnerability scan may make use of different protocols) to any destination and port, and in the main firewall we allowed for all the traffic coming from Greenbone to for any kind of protocol to any destination and port as well. This enables Greenbone to do a vulnerability scan to all the hosts in the ACME network.



8. All network hosts must be managed via SSH only from hosts within the Client network, so be sure that all the hosts have the SSH service up and running

Created two rules on the firewall to block connections to ssh from networks that are not the client network.

1) Main firewall->rules->floating, block connections on the “in” direction that come from DMZ, External services, WAN that have port 22 as destination.
2) Internal firewall->rules->SERVERS, block connections on the “in” direction that come from the internal servers network and have port 22 as destination.

Now only the client network should be able to initiate ssh connections.

We finally verified that all the hosts had SSH up and running.

```
root@arpwatch-clients:~# systemctl status ssh
* ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2025-06-16 13:25:21 UTC; 2 days ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 123 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 136 (sshd)
      Tasks: 1 (limit: 618872)
     Memory: 2.1M
        CPU: 23ms
     CGroup: /system.slice/ssh.service
             `-136 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups

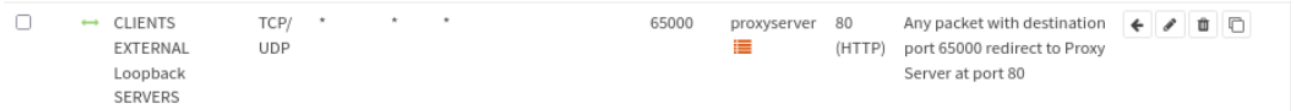
Jun 16 13:25:21 arpwatch-clients systemd[1]: Starting OpenBSD Secure Shell server...
Jun 16 13:25:21 arpwatch-clients sshd[136]: Server listening on 0.0.0.0 port 22.
Jun 16 13:25:21 arpwatch-clients sshd[136]: Server listening on :: port 22.
Jun 16 13:25:21 arpwatch-clients systemd[1]: Started OpenBSD Secure Shell server.
```

9. The Client network hosts have only access to external web services (HTTP/HTTPS) through the proxy server in the DMZ.

We make the Client network hosts reach the external web services (that are, the web services on the nets outside the internal firewall, i.e., Web server in the DMZ and Fantastic coffe in the External services) through a transparent proxy (for HTTP) implemented by the Proxy server in the DMZ, and blocking the HTTPS traffic that is not directed to the Proxy server.

10. Any packet the Main Firewall receives on port 65000 should be redirected to port 80 of the proxy host.

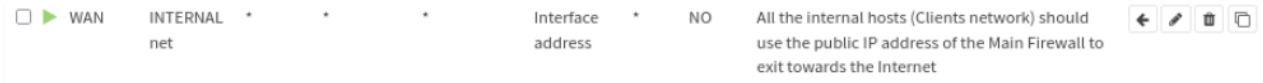
Set a rule in Main Firewall on the Firewall > NAT > Port Forward tab that forward all packets with destination port is 65000 to Proxy Server on port 80.



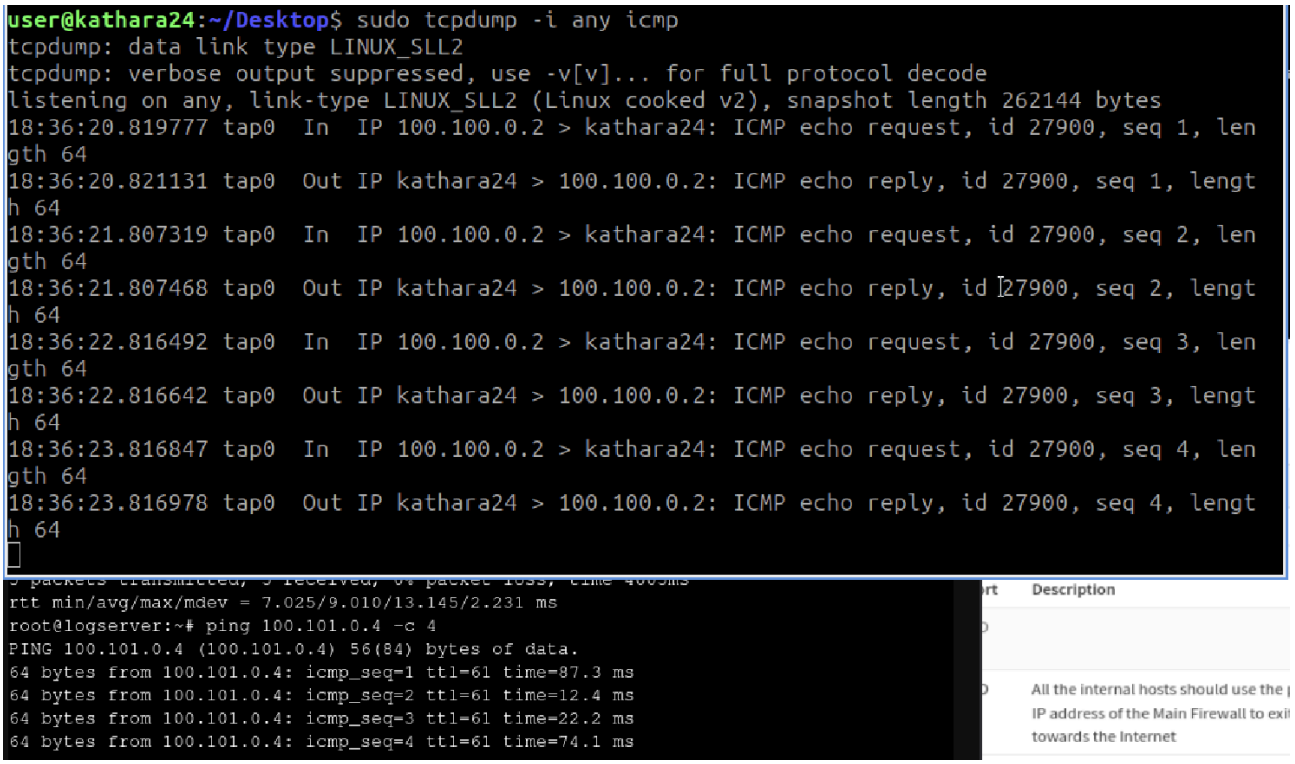
11. All the internal hosts should use the public IP address of the Main Firewall to exit towards the Internet.

Go to Firewall > NAT > outbound and select **Manual Outbound NAT rule generation (Advanced Outbound NAT (AON)).**

Now we set a new rule:



We finally test it and we get what can be seen below.

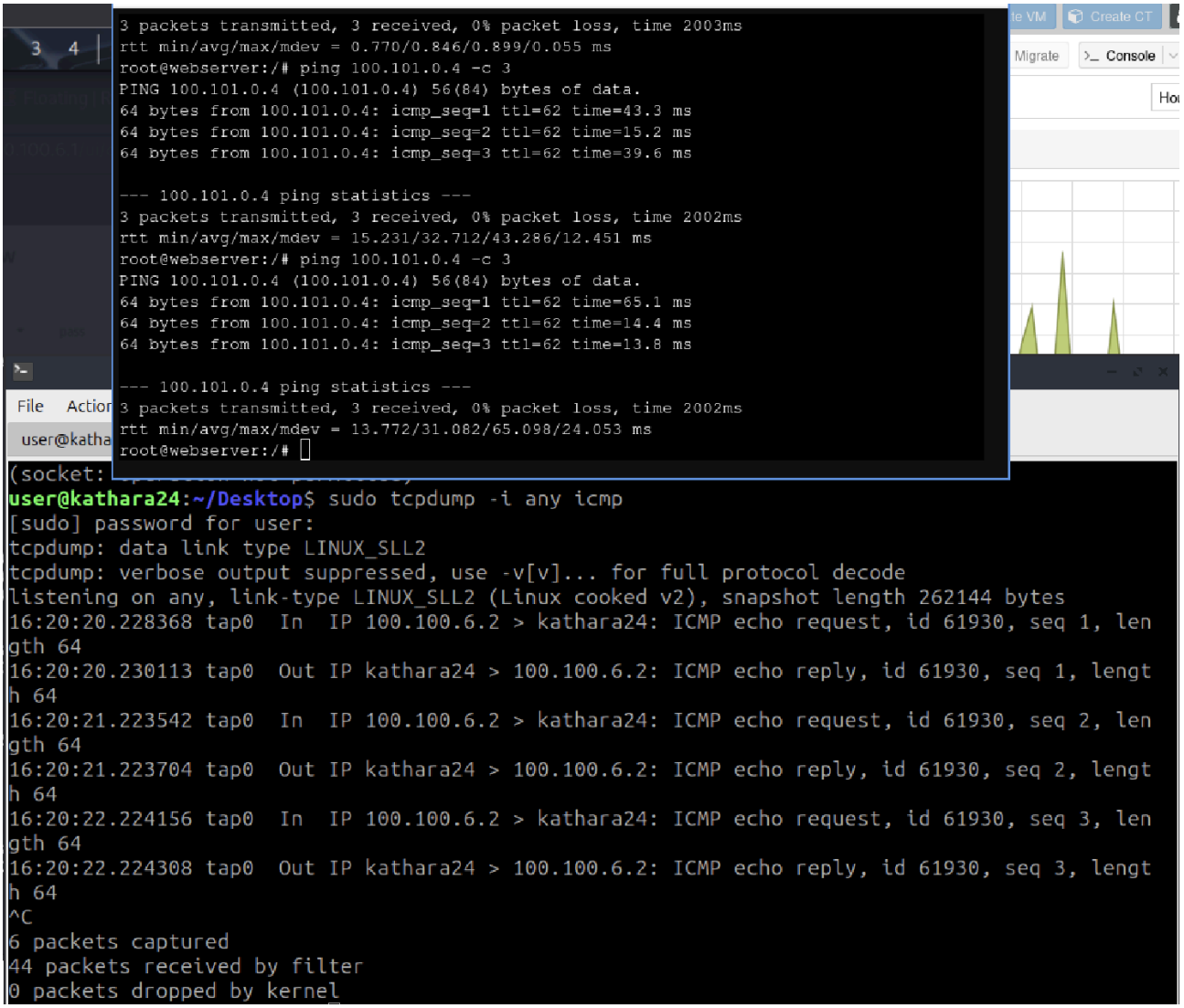


Because an https transparent proxy is not feasible, I deleted the rule that forwards tcp traffic from clients_network 443 to proxy 3128. Clients need to use the https proxy willingly. So I added a rule that allows clients_network to connect to the proxy server. on 3128.

12. All the hosts of the ACME network should be able to ping (and receive replies of) the other hosts and the Internet hosts.

Added a rule on internal_firewall->rules->floating and on main_firewall->rules->floating that allows all ICMP packets.

Modified port forwarding on main firewall. The destination was “this firewall”, this option means all addresses of the firewall. Modified to “Wan address” so that it only includes firewall addresses in the wan interface, like it’s specified in the assignment. Added debug rule to let me in the network with my personal vm.



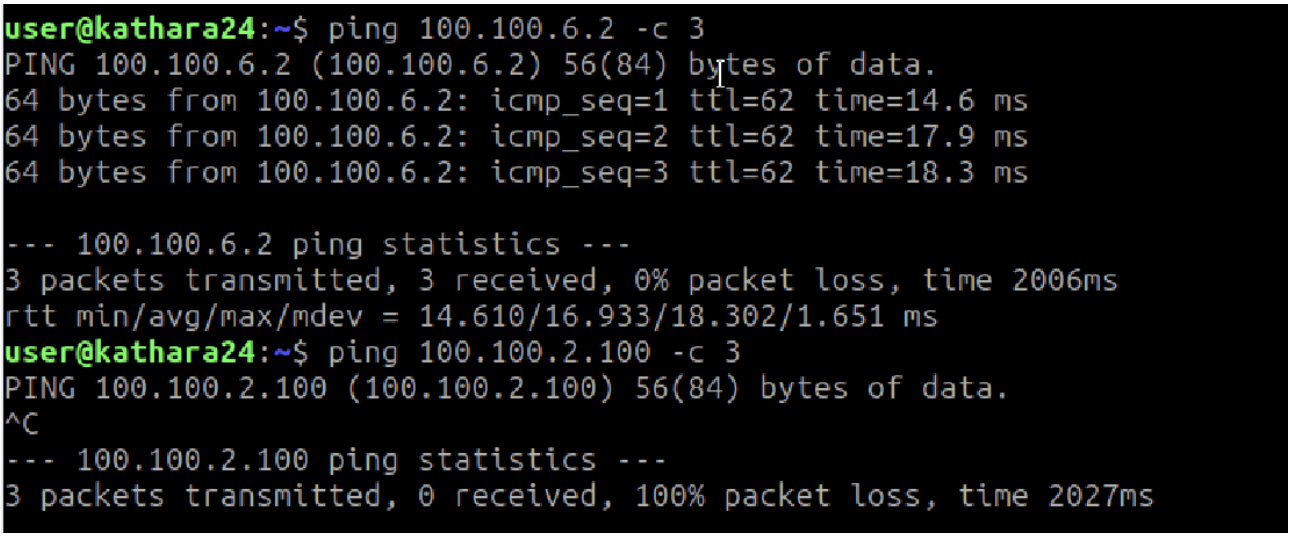
13. Only hosts in the DMZ should be reachable using the ping and traceroute tools from the Internet

added a firewall rule with ICMP protocol with source WAN net and destination DMZ.

Traceroute has to be done in order to select the right ports and not allow UDP traffic on all the ports, so we select option “other” and select ports 33434-33454 Make sure when creating rules under advanced settings that the state is set to “keep state”.

Bear in mind that allowing for UDP traffic from the WAN to the DMZ indiscriminately leads to a potential abuse of the protocol for attacks like UDP scanning and it increases attack surface. Nevertheless, we cannot restrict further, because some implementations of traceroute (especially on Linux and other Unix systems) rely on raw UDP packets to work properly.

We tested these firewall rules by pinging (and tracerouting) the webserver from outside the ACME network using our Lubuntu VM. As it can be seen, the webserver is ping-able (and traceroute-able), while the host in the Clients network is not.



14. ICMP redirect packets should not cross any network.

We can use two options to do it: set a rule in the two firewalls or disable ICMP redirects in system settings. For ease we chose the second one.

Go to System > Settings > Tunables and set these parameters:

- net.inet.icmp.drop_redirect = 1
- net.inet.ip.redirect = 0

We did this on both firewalls.

15. Anything that is not explicitly allowed has to be denied.

We were thinking that this rule changes our approach in general. We need to deny everything by default and then allow what is allowed. So no rules that just allow traffic on any protocol or any port. So I deleted the rule on internal_firewall->rules->clients that allows all ipv4+6 traffic in the “in” direction. Then added a rule in the internal firewall->rules->floating that allows all traffic on udp 53 to the dns, and all traffic on tcp 80/443 to the webserver.

Test of configuration

The tests of configuration are done and explained during the implementation of firewalls rules in the chapter above.

Final Remark

Since anything that is not explicitly allowed has to be denied, I modified the ssh rules. Deleted the floating rules in both firewalls related to that and allowed ssh connections coming from clients in internal_firewall->rules->CLIENTS

Final adjustments. Hosts from the DMZ should be isolated from the internal network(i.e. int_net can initiate connections to DMZ but not the opposite). Waiting on others to see what they think on modifying rule on int_fw->rules->external that allows all “in” traffic.

Right now the configuration to block traffic to INT_SERVERS_WITHOUT_DNS with a source different from clients and DMZ does not work. This is because of the automatically generated rule that allows all out traffic on servers_out. Instead of blocking connections at the out of servers, we can allow them at in from clients and at in from external, so that everything else is blocked automatically and it’s cleaner this way.